

本节内容

栈的应用

——括号匹配

括号匹配问题

```
void test() {  
    int a[10][10];  
    int x = 10*(20*(1+1)-(3-2));  
    printf("加油! 奥利给! ");  
}
```

Expected ')' to match this '('

```
void test() {  
    int a[10][10];  
    int x = 10*(20*(1+1)-(3-2));  
    printf("加油! 奥利给! ");  
}
```

每一个 单身的人 得看透

想爱 就别怕伤痛

找一个 最爱的 深爱的

想爱的 亲爱的人 来告别单身

又常常羡慕
别人成双入对



括号匹配问题



(((())))
① ② ③ ④ ④ ③ ② ①

最后出现的左括号最先被匹配 (LIFO)

可用“栈”
实现该特性

((())) ())
① ② ③ ③ ② ④ ④ ①

每出现一个右括号，就“消耗”一个左括号

出栈

算法演示

栈底



{ (()) [] }

① ② ③ ③ ② ④ ④ ①

所有括号都能两两配对

遇到左括号就入栈
遇到右括号，就“消耗”一个左括号

算法演示

栈底



{ (()] [] }

① ② ③ ③ ②

当前扫描到的右括号
与栈顶左括号不匹配

遇到左括号就入栈
遇到右括号，就“消耗”一个左括号

算法演示

栈底



{ (()) }] ()

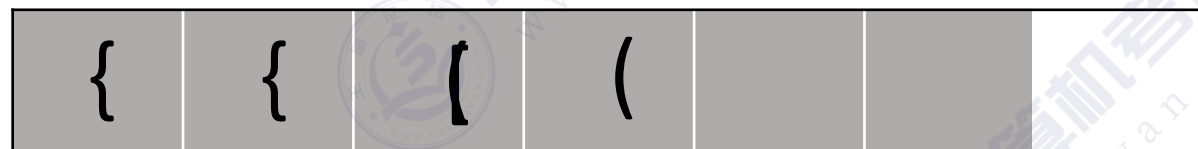
① ② ③ ③ ② ①

扫描到右括号且栈空，——右括号单身

遇到左括号就入栈
遇到右括号，就“消耗”一个左括号

算法演示

栈底



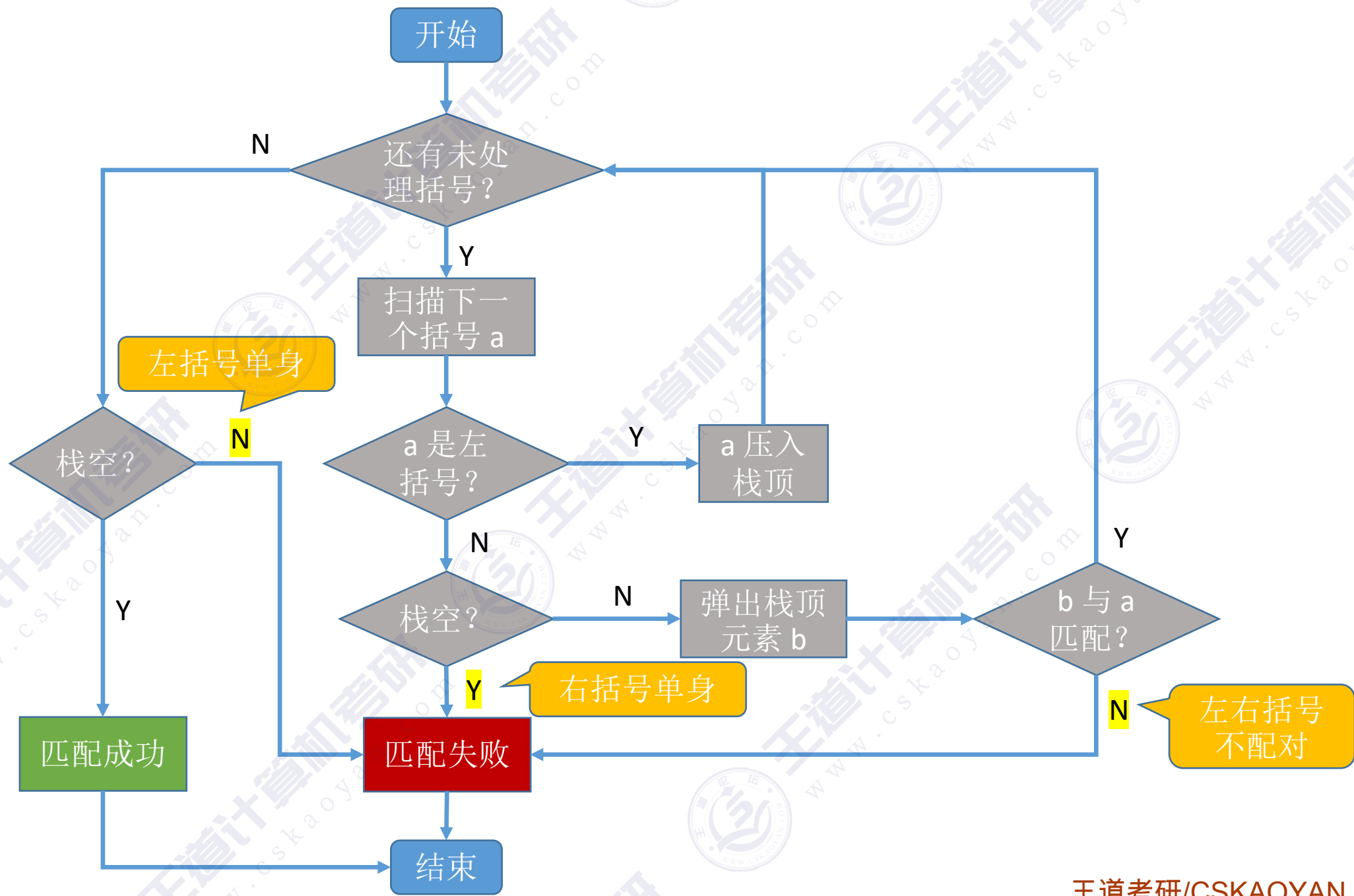
处理完所有括号后，栈非空——左括号单身

{ { (()) [] }

① ② ③ ④ ④ ③ ⑤ ⑤ ②

遇到左括号就入栈

遇到右括号，就“消耗”一个左括号



算法实现

万一存满了
可咋整？

可用链栈！

```
bool bracketCheck(char str[], int length) {
    SqStack S;
    InitStack(S); //初始化一个栈
    for (int i=0; i<length; i++){
        if (str[i]=='(' || str[i]=='[' || str[i]=='{'){
            Push(S, str[i]); //扫描到左括号，入栈
        } else {
            if (StackEmpty(S)) //扫描到右括号，且当前栈空
                return false; //匹配失败

            char topElem;
            Pop(S, topElem); //栈顶元素出栈
            if(str[i]==')' && topElem!='(')
                return false;
            if(str[i]==']' && topElem!='[')
                return false;
            if(str[i]=='}' && topElem!='{')
                return false;
        }
    }
    return StackEmpty(S); //检索全部括号后，栈空说明匹配成功
}
```

```
#define MaxSize 10 //定义栈中元素的最大个数
typedef struct{
    char data[MaxSize]; //静态数组存放栈中元素
    int top; //栈顶指针
} SqStack;
```

考试中可直接使用基本操作，建议简要说明接口

```
//初始化栈
void InitStack(SqStack &S)

//判断栈是否为空
bool StackEmpty(SqStack S)

//新元素入栈
bool Push(SqStack &S, char x)

//栈顶元素出栈，用x返回
bool Pop(SqStack &S, char &x)
```

练习：不要使用基本操作，
动手实现完整代码

知识回顾与重要考点

用栈实现**括号匹配**:

依次扫描所有字符，遇到左括号入栈，遇到右括号则弹出栈顶元素检查是否匹配。

匹配失败情况:

①左括号单身②右括号单身③左右括号不匹配

本节内容

栈的应用

——表达式求值

知识总览



大家熟悉的算数表达式



Reference: Wikipedia
——Reverse Polish notation

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$

③ ② ① ④ ⑦ ⑥ ⑤

$$15 \div 7 - 1 + 1 \times 3 - 2 + 1 + 1$$

① ② ④ ③ ⑤ ⑥ ⑦

由三个部分组成：操作数、运算符、界限符

界限符是必不可少的，
反映了计算的先后顺序

波兰数学家的灵感

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$



一个灵感：可以不用界限符也能无歧义地表达运算顺序

Reverse Polish notation（逆波兰表达式 = 后缀表达式）

Polish notation（波兰表达式 = 前缀表达式）

中缀、后缀、前缀表达式

运算符在两个
操作数中间

中缀表达式

$a + b$

$a + b - c$

$a + b - c * d$

规则：运算符在
两个操作数后面

后缀表达式

$ab +$

$ab + c -$

$ab + cd * -$

规则：运算符在
两个操作数前面

前缀表达式

$+ ab$

$- + abc$

$- + ab * cd$

中缀表达式转后缀表达式（手算）

中缀转后缀的手算方法：

- ① 确定中缀表达式中各个运算符的运算顺序
- ② 选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数
- ③ 如果还有运算符没被处理，就继续 ②

$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$

中缀表
达式

15 7 1 1 ^①+ ^②- ^③÷ 3 ^④× 2 1 1 ^⑤+ ^⑥+ ^⑦-

后缀表
达式

中缀表达式转后缀表达式（手算）

中缀转后缀的手算方法：

- ① 确定中缀表达式中各个运算符的运算顺序
- ② 选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数
- ③ 如果还有运算符没被处理，就继续 ②

运算顺序不唯一，因此对应的
后缀表达式也不唯一

$A + B * (C - D) - E / F$

③ ② ① ⑤ ④



① ② ③ ④ ⑤

A B C D - * + E F / -

$A + B * (C - D) - E / F$

⑤ ③ ② ④ ①



② ③ ① ④ ⑤

A B C D - * E F / - +

私房菜：“左优先”原则，不要FreeStyle，保证手算和机算结果相同

“左优先”原则：只要左边的运算符能先计算，就优先算左边的

客观来看两种都正确，只是“机算”结果是前者

中缀表达式转后缀表达式（手算）

中缀转后缀的手算方法：

- ① 确定中缀表达式中各个运算符的运算顺序
- ② 选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数
- ③ 如果还有运算符没被处理，就继续 ②

运算顺序不唯一，因此对应的
后缀表达式也不唯一

“左优先”原则：只要左边的运算符能先计算，就优先算左边的

可保证运算顺序唯一

$A + B - C * D / E + F$

① ④ ② ③ ⑤

$A B + C D * E / - F +$

① ② ③ ④ ⑤

后缀表达式的计算（手算）

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$

③

②

①

④

⑦

⑥

⑤

中缀表达式

$$15 \quad 7 \quad 1 \quad 1 \quad + \quad - \quad \div \quad 3 \quad \times \quad 2 \quad 1 \quad 1 \quad + \quad + \quad -$$

①

②

③

④

⑤

⑥

⑦

后缀表达式

后缀表达式的手算方法：

从左往右扫描，每遇到一个运算符，就让运算符前面最近的两个操作数执行对应运算，合体为一个操作数

注意：两个操作数的左右顺序

后缀表达式的计算（手算）

后缀表达式的手算方法：

从左往右扫描，每遇到一个运算符，就让运算符前面最近的两个操作数执行对应运算，合体为一个操作数

注意：两个操作数的左右顺序

$$\begin{array}{ccccccccc} A & + & B & * & (C & - & D) & - & E / F \\ \textcircled{3} & & \textcircled{2} & & \textcircled{1} & & \textcircled{5} & & \textcircled{4} \end{array}$$

$$\begin{array}{ccccccccc} A & B & C & D & - & * & + & E & F & / & - \\ & & \textcircled{1} & \textcircled{2} & \textcircled{3} & & & \textcircled{4} & \textcircled{5} & & \\ & \underbrace{\hspace{1.5cm}} & & & & & & & & & \\ & \underbrace{\hspace{2.5cm}} & & & & & & & & & \\ & \underbrace{\hspace{4.5cm}} & & & & & \underbrace{\hspace{1.5cm}} & & & & \end{array}$$

后缀表达式的计算（手算）

后缀表达式的手算方法：

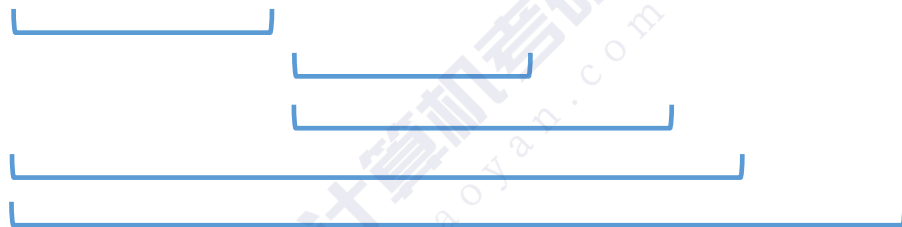
从左往右扫描，每遇到一个运算符，就让运算符前面最近的两个操作数执行对应运算，合体为一个操作数

注意：两个操作数的左右顺序

$A + B - C * D / E + F$

① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $AB + CD * E / - F +$



认真思考

耶！



LIFO（后进先出）

栈！！

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

$A + B - C * D / E + F$

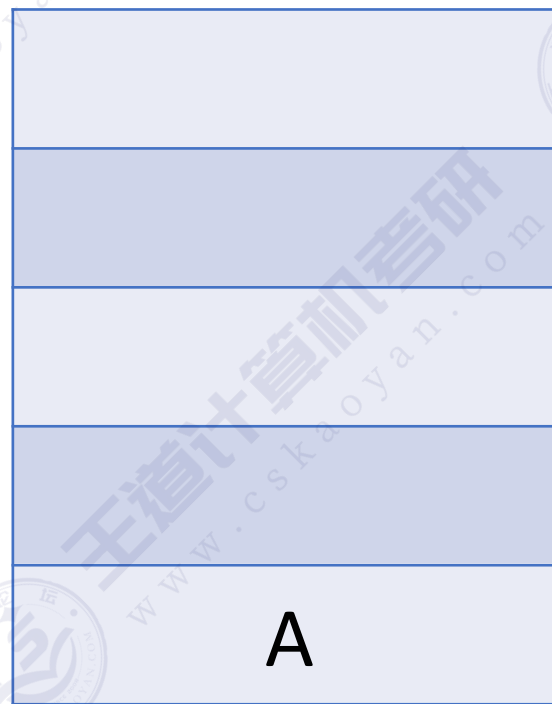
① ④ ② ③ ⑤

① ② ③ ④ ⑤

$A B + C D * E / - F +$



栈



后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

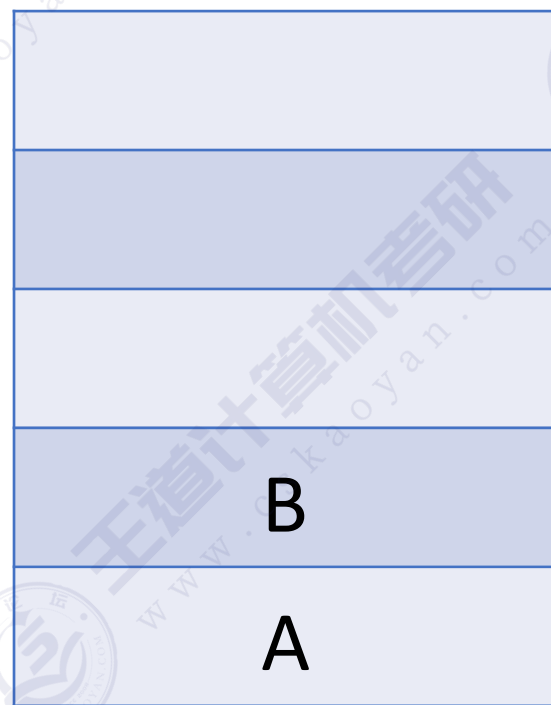
$A + B - C * D / E + F$

① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $A B + C D * E / - F +$



栈



后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

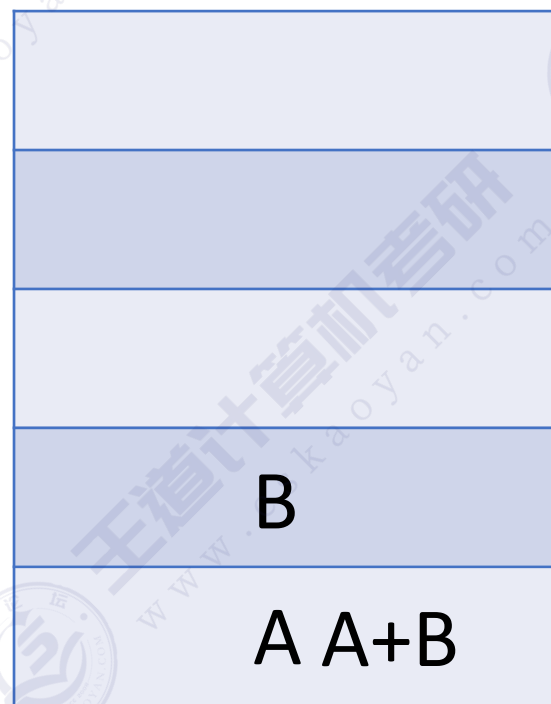
$A + B - C * D / E + F$

① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $A B + C D * E / - F +$



栈



注意：先出栈的是“右操作数”

+

后缀表达式的计算（机算）

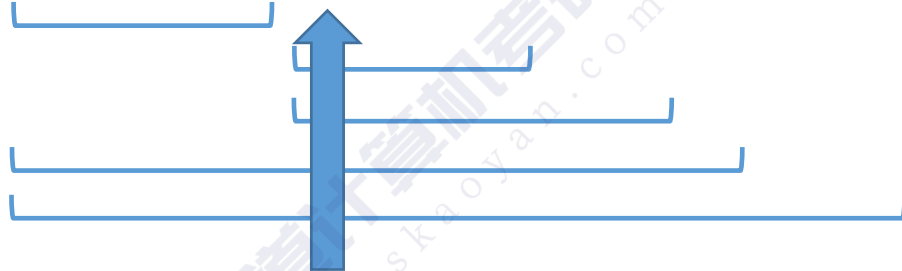
用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

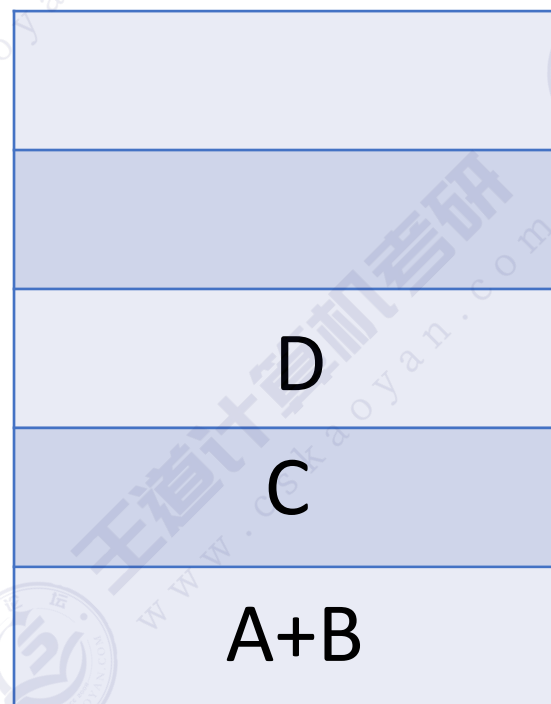
$A + B - C * D / E + F$

① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $A \ B \ + \ C \ D \ * \ E \ / \ - \ F \ +$



栈



$C * D$

后缀表达式的计算（机算）

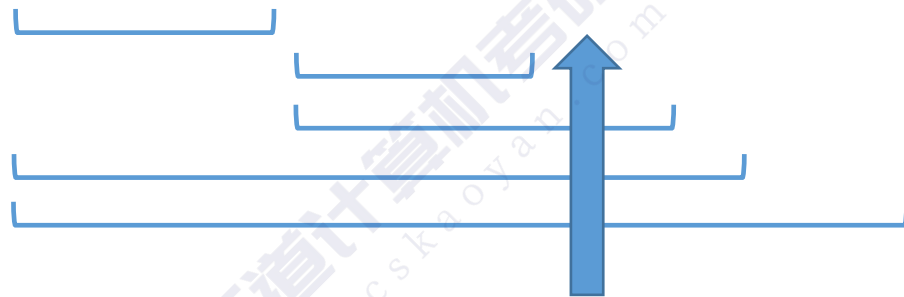
用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

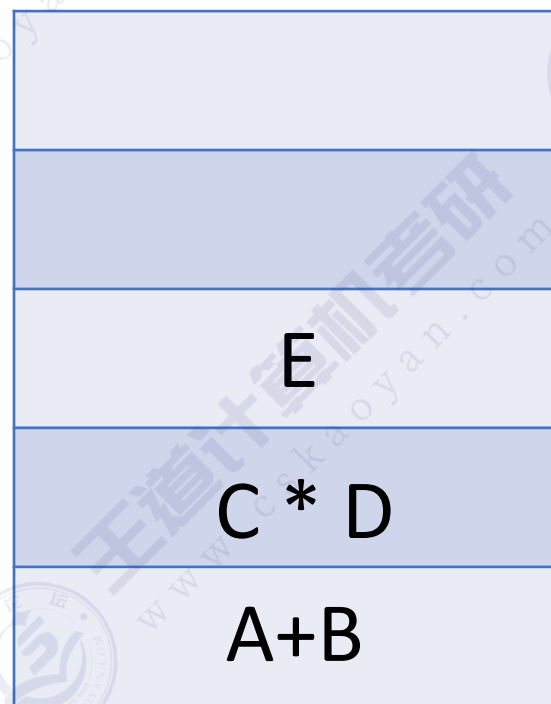
A + B - C * D / E + F

① ④ ② ③ ⑤

① ② ③ ④ ⑤
A B + C D * E / - F +



栈



$(C * D) / E$

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

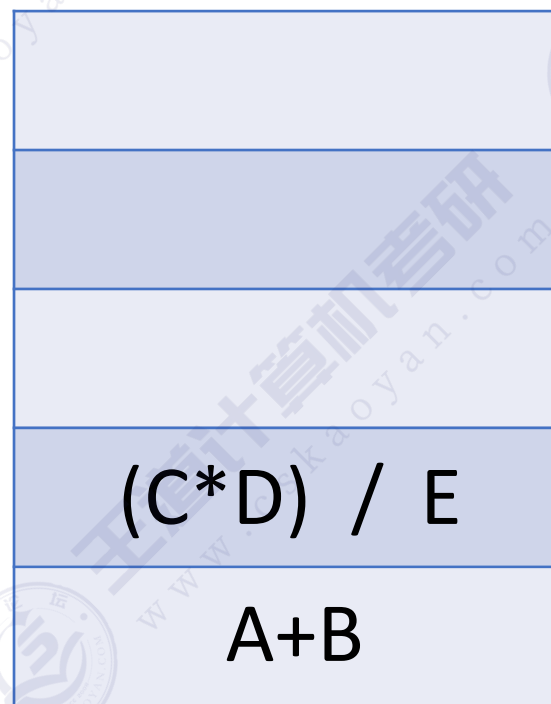
A + B - C * D / E + F

① ④ ② ③ ⑤

① ② ③ ④ ⑤
A B + C D * E / - F +



栈



$(A+B)-((C*D)/E)$

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

A + B - C * D / E + F

①

④

②

③

⑤

①

②

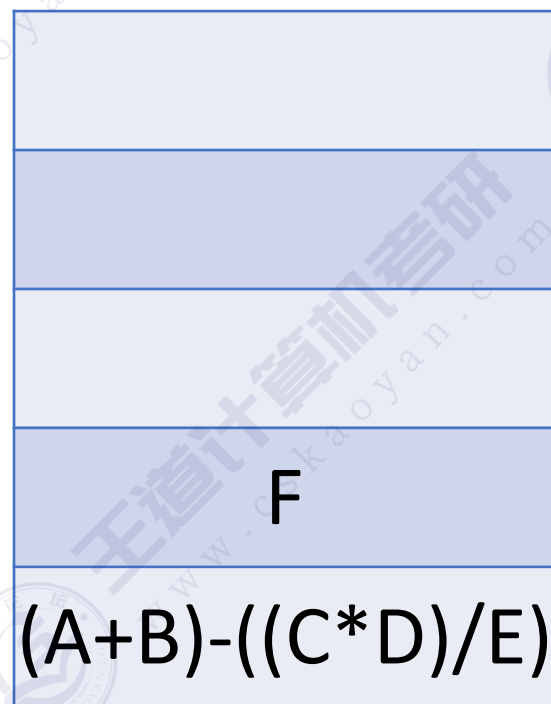
③

④

⑤

A B + C D * E / - F +

栈



((A+B)-((C*D)/E))+F

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

注意：先出栈的是“右操作数”

A + B - C * D / E + F

① ④ ② ③ ⑤

① ② ③ ④ ⑤
A B + C D * E / - F +



栈



若表达式合法，
则最后栈中只会
留下一个元素，
就是最终结果

后缀表达式的计算（机算）

后缀表达式适用于基于栈的编程语言（**stack-oriented programming language**），如：
Forth、PostScript

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

注意：先出栈的是“右操作数”

栈

$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$

③ ② ① ④ ⑦ ⑥ ⑤

15 7 1 1 + - ÷ 3 × 2 1 1 + + -

思考：后缀表达式怎么转中缀？

放弃思考



中缀表达式转前缀表达式（手算）

中缀转前缀的手算方法：

- ① 确定中缀表达式中各个运算符的运算顺序
- ② 选择下一个运算符，按照「运算符 左操作数 右操作数」的方式组合成一个新的操作数
- ③ 如果还有运算符没被处理，就继续 ②

“右优先”原则：只要右边的运算符能先计算，就优先算右边的

$A + B * (C - D) - E / F$

③

②

①

⑤

④

$A + B * (C - D) - E / F$

⑤

③

②

④

①

$- + A * B - C D / E F$

⑤

③

②

①

④

$+ A - * B - C D / E F$

⑤

④

③

②

①

中缀表达式转前缀表达式（手算）

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$

③ ② ① ④ ⑦ ⑥ ⑤

中缀转后缀：
“左优先”

15 7 1 1 + - ÷ 3 × 2 1 1 + + -

$$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$$

⑤ ④ ③ ⑥ ⑦ ② ①

中缀转前缀：
“右优先”

- × ÷ 15 - 7 + 1 1 3 + 2 + 1 1

⑦ ⑥ ⑤ ④ ③ ② ①

前缀表达式的计算

用栈实现前缀表达式的计算：

- ① 从右往左扫描下一个元素，直到处理完所有元素
 - ② 若扫描到操作数则压入栈，并回到①；否则执行③
 - ③ 若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①
- 注意：先出栈的是“左操作数”
- 栈

$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$

⑤ ④ ③ ⑥ ⑦ ② ①

- × ÷ 15 - 7 + 1 1 3 + 2 + 1 1

⑦ ⑥ ⑤ ④ ③ ② ①



知识回顾与重要考点

概念 ⊖ 运算符、操作数、界限符 (DIY概念: 左操作数/右操作数)

三种表达式 ⊖

中缀表达式 ⊖ 运算符在操作数中间

后缀表达式 (逆波兰式) ⊖ 运算符在操作数后面

前缀表达式 (波兰式) ⊖ 运算符在操作数前面

一个中缀表达式可以对应多个后缀、前缀表达式

一个中缀表达式只对应一个后缀表达式 (确保算法的“确定性”)

表达式求值问题



后缀表达式考点

中缀转后缀 ⊖

①按“左优先”原则确定运算符的运算次序

②根据①中确定的次序, 依次将各个运算符和与之相邻的两个操作数按<左操作数 右操作数 运算符>的规则合体

后缀转中缀 ⊖

从左往右扫描, 每遇到一个运算符, 就将<左操作数 右操作数 运算符>变为(左操作数 运算符 右操作数)的形式

计算 ⊖

从左往右扫描, 遇到操作数入栈, 遇到运算符则弹出两个栈顶元素运算后入栈 (注意: 先弹出的元素是“右操作数”)

①按“右优先”原则确定运算符的运算次序

②根据①中确定的次序, 依次将各个运算符和与之相邻的两个操作数按<运算符 左操作数 右操作数>的规则合体

前缀表达式 ⊖

计算 ⊖

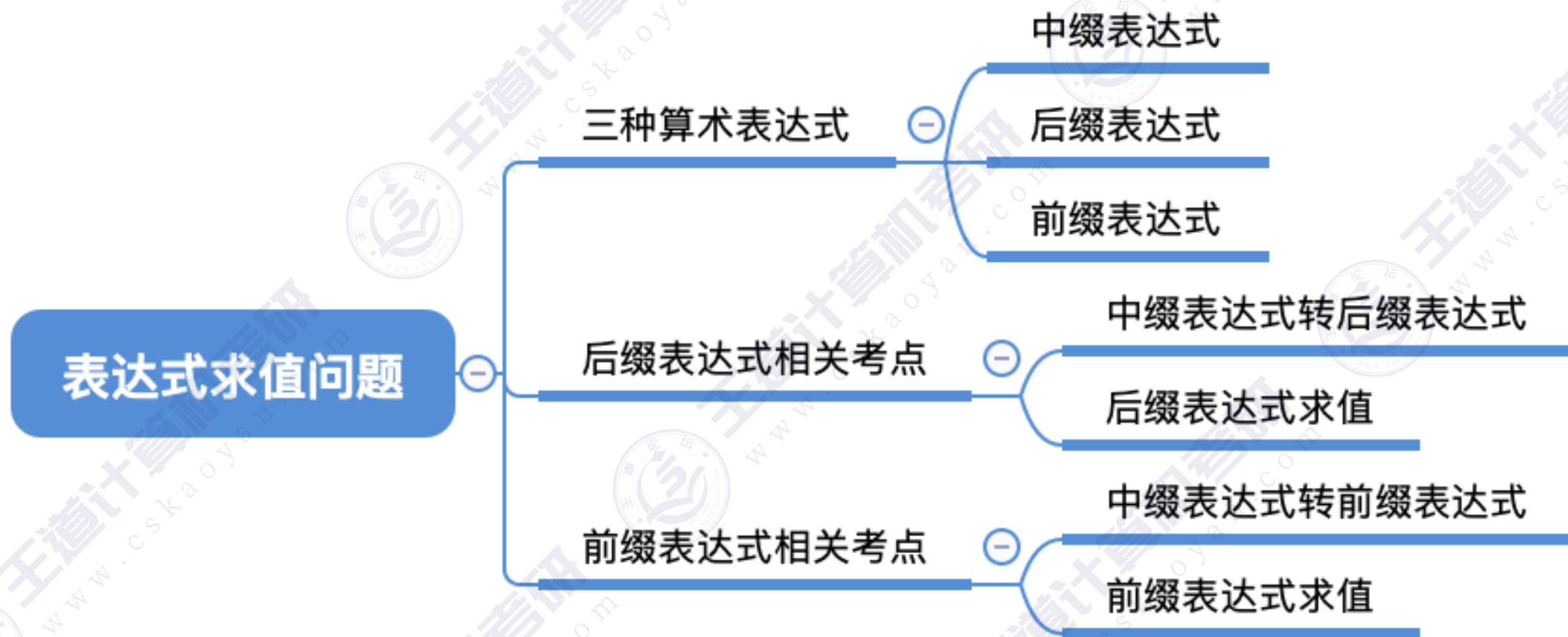
从右往左扫描, 遇到操作数入栈, 遇到运算符则弹出两个栈顶元素运算后入栈 (注意: 先弹出的元素是“左操作数”)

本节内容

栈的应用

——表达式求值

知识总览



知识总览

表达式求值问题（第二季）

中缀表达式转后缀表达式（机算，用栈实现）

中缀表达式的计算（用栈实现）

中缀表达式转后缀表达式（手算）

中缀转后缀的手算方法：

- ① 确定中缀表达式中各个运算符的运算顺序
- ② 选择下一个运算符，按照「左操作数 右操作数 运算符」的方式组合成一个新的操作数
- ③ 如果还有运算符没被处理，就继续 ②

“左优先”原则：只要左边的运算符能先计算，就优先算左边的

$A + B - C * D / E + F$

①

④

②

③

⑤

$AB + CD * E / - F +$

①

②

③

④

⑤

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中**优先级**高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

*** / 优先级高于 + -**

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④

②

③

⑤

A B + C D * E / - F +

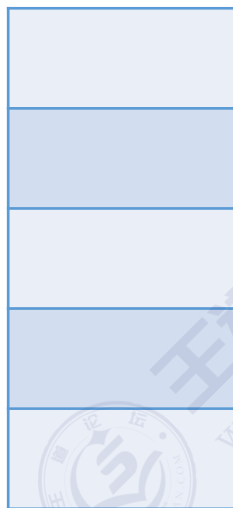
①

②

③

④

⑤



A

中缀表达式转后缀表达式（机算）

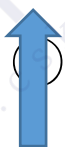
初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F



④

②

③

⑤

A B + C D * E / - F +

①

②

③

④

⑤



A

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①



④

②

③

⑤

A B + C D * E / - F +

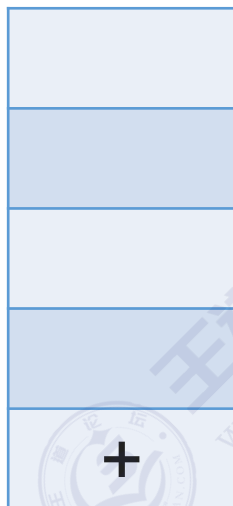
①

②

③

④

⑤



A B

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①



②

③

⑤

A B + C D * E / - F +

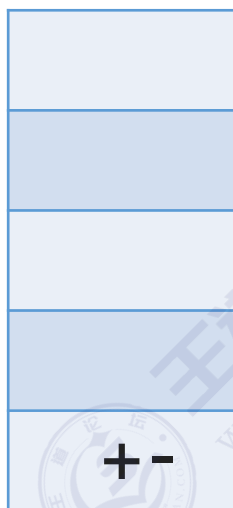
①

②

③

④

⑤



A B

对比：“左优先”原则

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④

②

③

⑤

A B + C D * E / - F +

①

②

③

④

⑤



A B + C

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

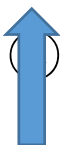
- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④



③

⑤

A B + C D * E / - F +

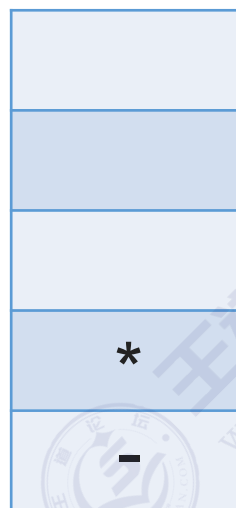
①

②

③

④

⑤



A B + C

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到“)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④

②

③

⑤



A B + C D * E / - F +

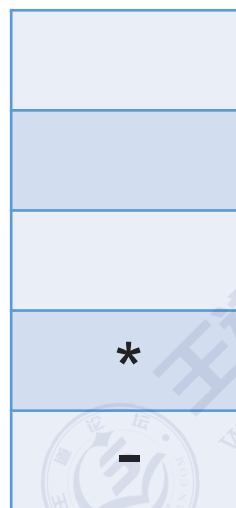
①

②

③

④

⑤



A B + C D

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④

②



⑤

A B + C D * E / - F +

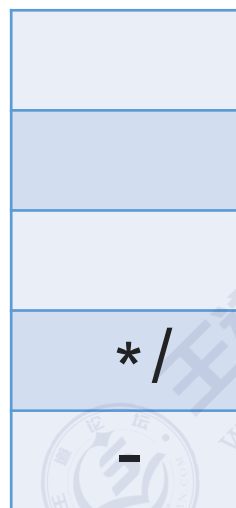
①

②

③

④

⑤



A B + C D

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④

②

③

⑤



A B + C D * E / - F +

①

②

③

④

⑤



A B + C D * E

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④

②

③



A B + C D * E / - F +

①

②

③

④

⑤



A B + C D * E

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④

②

③

⑤



A B + C D * E / - F +

①

②

③

④

⑤



A B + C D * E / - F

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④

②

③

⑤



A B + C D * E / - F +

①

②

③

④

⑤



A B + C D * E / - F +

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B * (C - D) - E / F

③ ② ① ⑤ ④

① ② ③ ④ ⑤
A B C D - * + E F / -



A

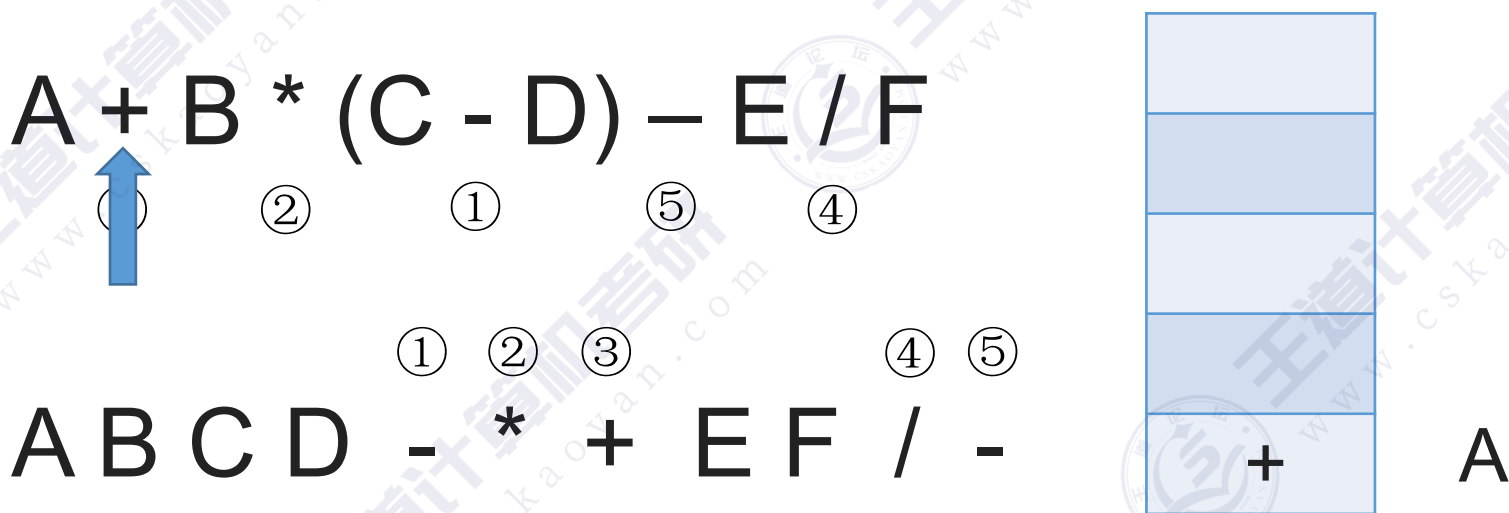
中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



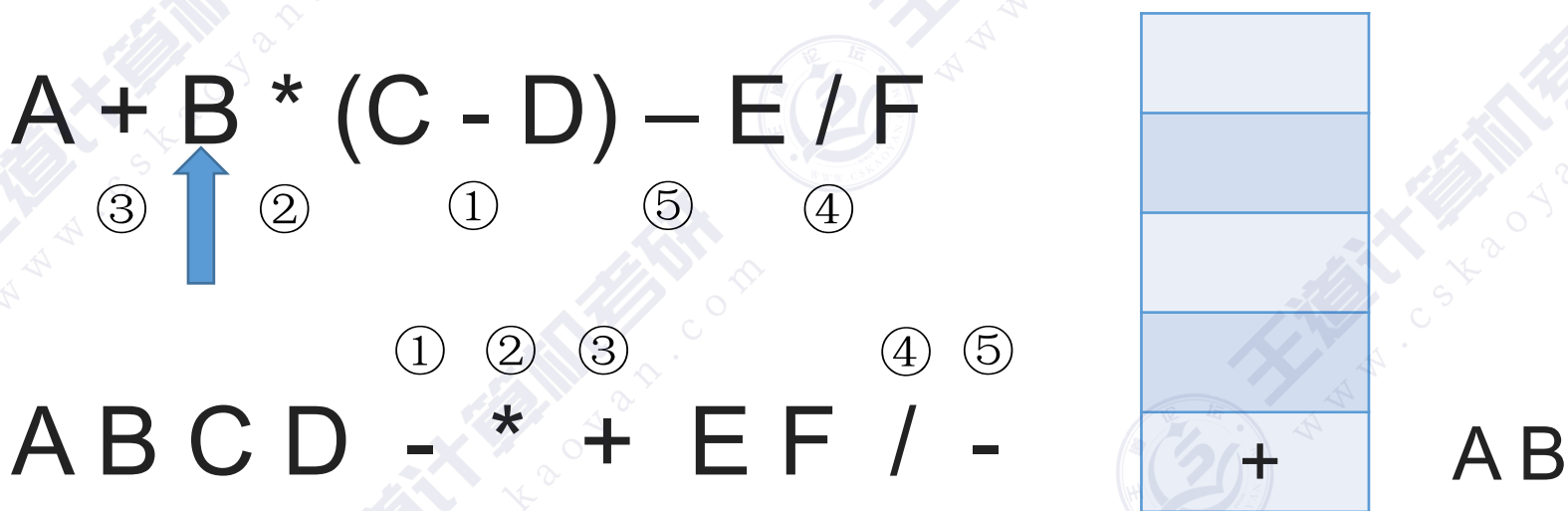
中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。



中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

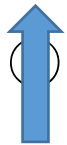
从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

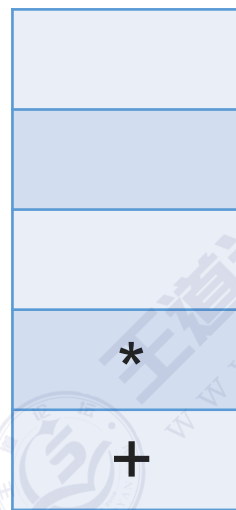
$A + B * (C - D) - E / F$

③ ① ⑤ ④



① ② ③ ④ ⑤

$A B C D - * + E F / -$



$A B$

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

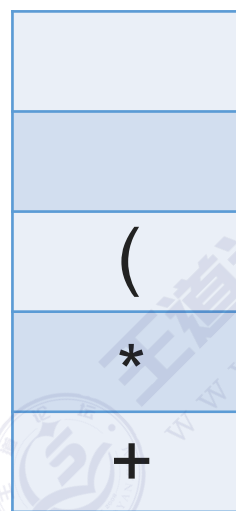
$A + B * (C - D) - E / F$

③ ② ① ⑤ ④



① ② ③ ④ ⑤

$A B C D - * + E F / -$



$A B$

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

$A + B * (C - D) - E / F$

③ ② ① ⑤ ④

① ② ③ ④ ⑤

A B C D - * + E F / -



A B C

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

$A + B * (C - D) - E / F$

③

②



⑤

④

①

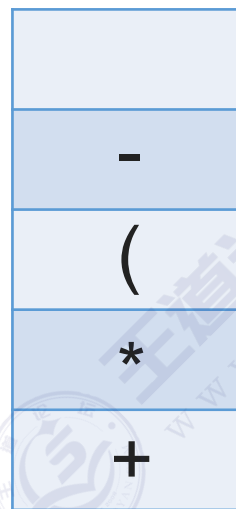
②

③

④

⑤

A B C D - * + E F / -



A B C

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

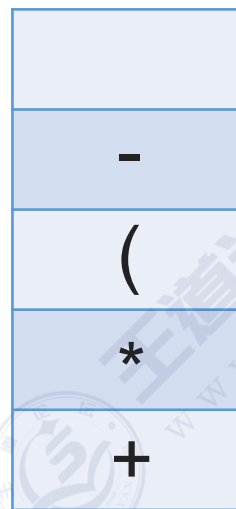
按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

$A + B * (C - D) - E / F$

③ ② ① ⑤ ④

① ② ③ ④ ⑤

A B C D - * + E F / -



A B C D

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

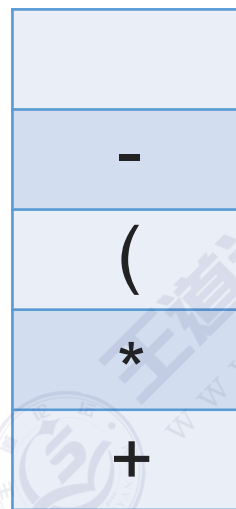
按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

$A + B * (C - D) - E / F$

③ ② ① ⑤ ④

① ② ③ ④ ⑤

A B C D - * + E F / -



A B C D

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B * (C - D) - E / F

③

②

①

④

⑤

①

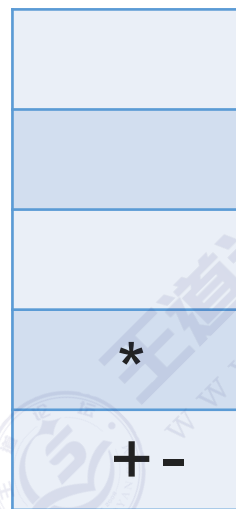
②

③

④

⑤

A B C D - * + E F / -



A B C D -

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

$A + B * (C - D) - E / F$

③ ② ① ⑤ ④



① ② ③ ④ ⑤

$A B C D - * + E F / -$



$A B C D - * + E$

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

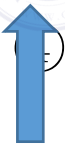
A + B * (C - D) - E / F

③

②

①

⑤



①

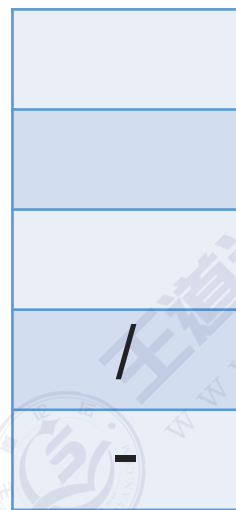
②

③

④

⑤

A B C D - * + E F / -



A B C D - * + E

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B * (C - D) - E / F

③

②

①

⑤

④



①

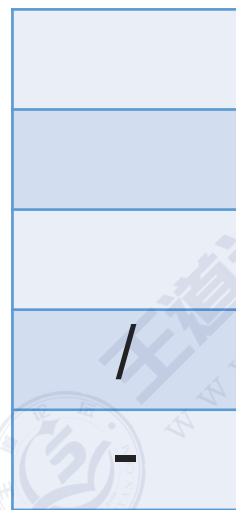
②

③

④

⑤

A B C D - * + E F / -



A B C D - * + E F

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

$A + B * (C - D) - E / F$

③ ② ① ⑤ ④

$A B C D - * + E F / -$



$A B C D - * + E F / -$

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

$((15 \div (7 - (1 + 1))) \times 3) - (2 + (1 + 1))$

③

②

①

④

⑦

⑥

⑤

15

7

1

1

①

+

②

-

③

÷

3

④

×

2

1

1

⑤

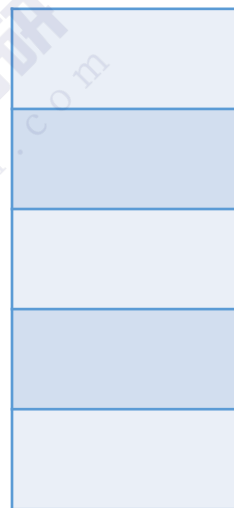
+

⑥

+

⑦

-



栈是否会
溢出？

中缀表达式的计算（用栈实现）

中缀转后缀
+
后缀表达式求值
两个算法的结合

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

$A + B - C * D / E + F$

① ④ ② ③ ⑤

① ② ③ ④ ⑤

$A B + C D * E / - F +$



栈



后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

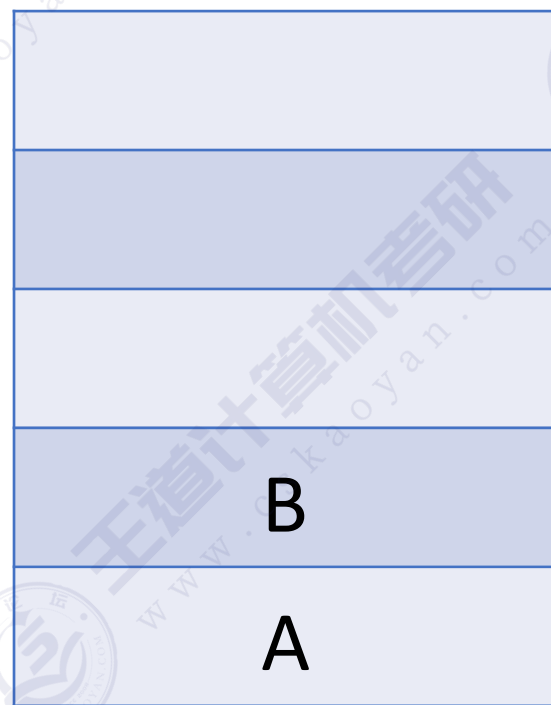
$A + B - C * D / E + F$

① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $A B + C D * E / - F +$



栈



后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

A + B - C * D / E + F

①

④

②

③

⑤

①

②

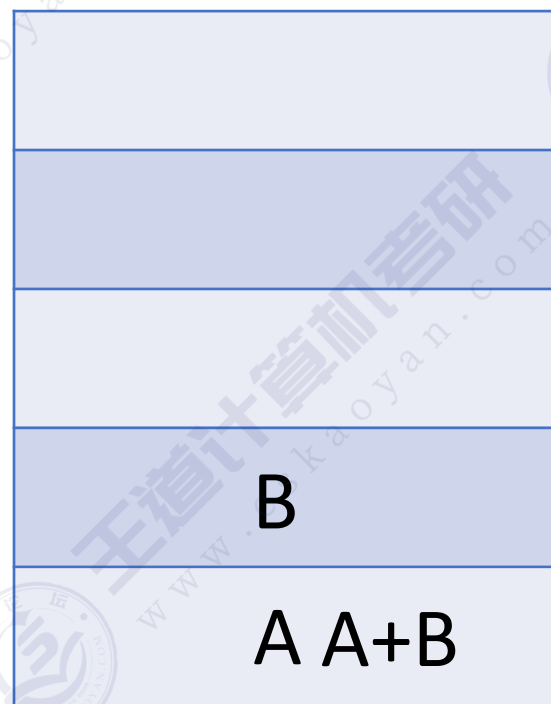
③

④

⑤

A B + C D * E / - F +

栈



注意：先出栈的是“右操作数”

+

后缀表达式的计算（机算）

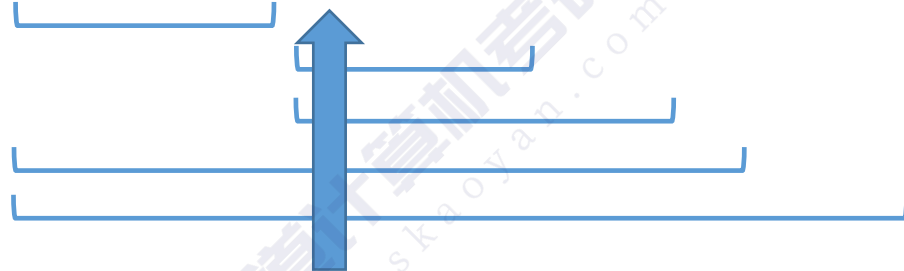
用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

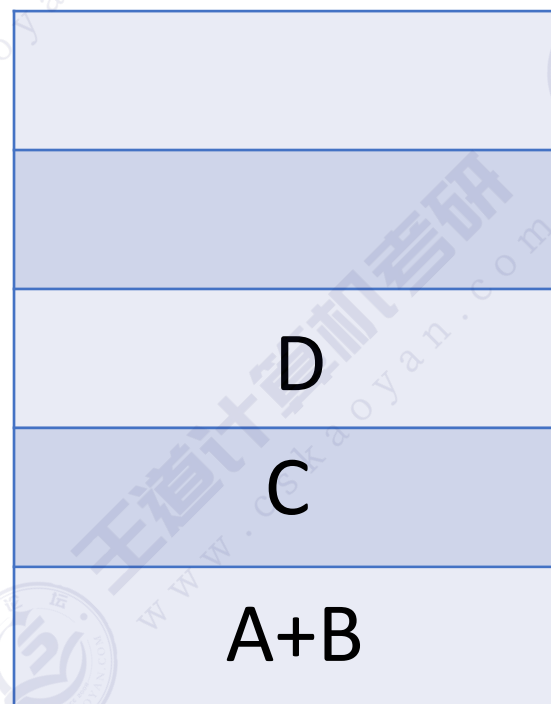
$A + B - C * D / E + F$

① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $A \ B \ + \ C \ D \ * \ E \ / \ - \ F \ +$



栈



$C * D$

后缀表达式的计算（机算）

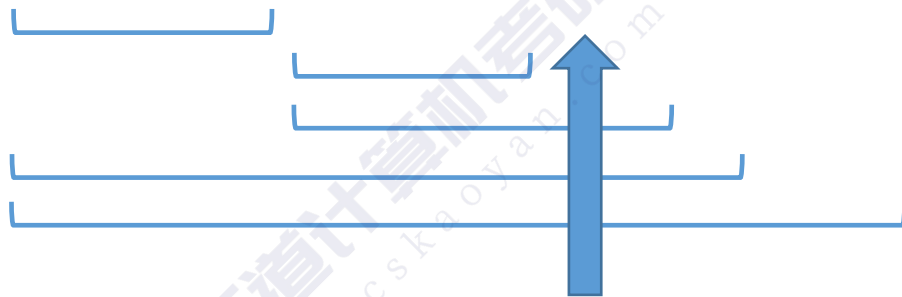
用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

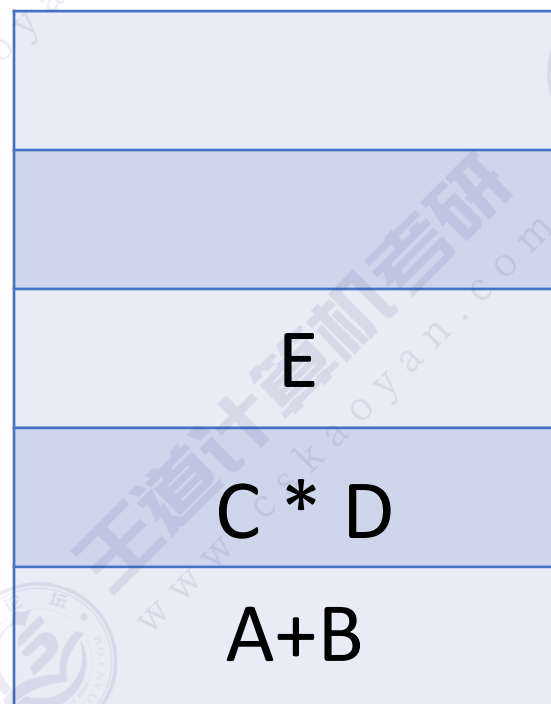
$A + B - C * D / E + F$

① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $A B + C D * E / - F +$



栈



$(C * D) / E$

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

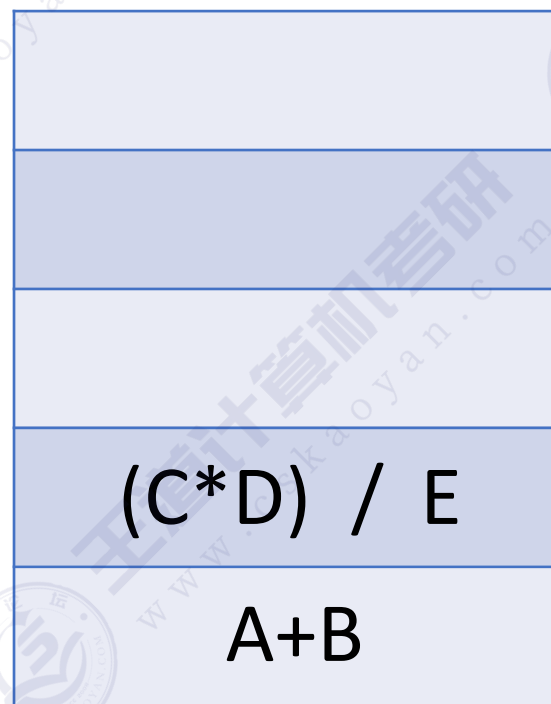
A + B - C * D / E + F

① ④ ② ③ ⑤

① ② ③ ④ ⑤
A B + C D * E / - F +



栈



$(A+B)-((C*D)/E)$

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

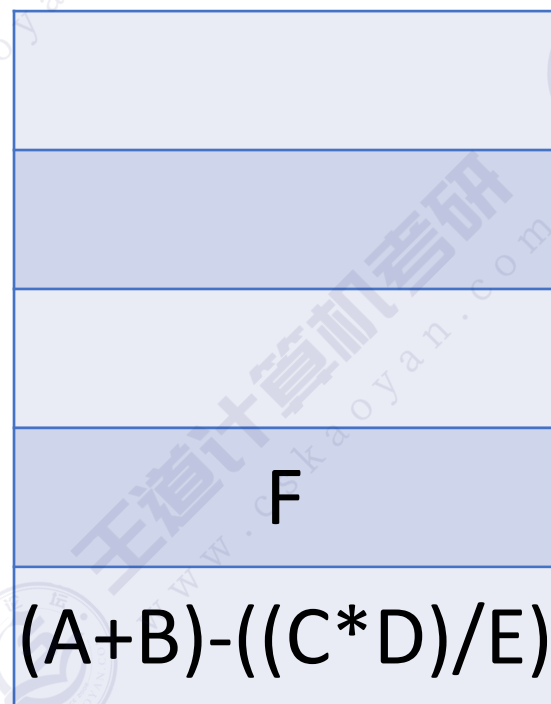
A + B - C * D / E + F

① ④ ② ③ ⑤

① ② ③ ④ ⑤
A B + C D * E / - F +



栈



$((A+B)-((C*D)/E))+F$

后缀表达式的计算（机算）

用栈实现后缀表达式的计算：

- ①从左往右扫描下一个元素，直到处理完所有元素
- ②若扫描到操作数则压入栈，并回到①；否则执行③
- ③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

注意：先出栈的是“右操作数”

$A + B - C * D / E + F$

① ④ ② ③ ⑤

① ② ③ ④ ⑤
 $A B + C D * E / - F +$



栈

用于存放当前暂时还不能确定运算次序的操作数

$((A+B)-((C*D)/E))+F$

中缀表达式转后缀表达式（机算）

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素，直到末尾。可能遇到三种情况：

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈；遇到”)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

A + B - C * D / E + F

①

④

②

③

⑤



A B + C D * E / - F +

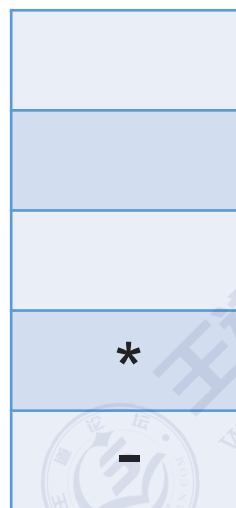
①

②

③

④

⑤



用于存放当前暂时还不能确定运算次序的运算符

A B + C D

中缀表达式的计算（用栈实现）

中缀转后缀
+
后缀表达式求值
两个算法的结合

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

①

④

②

③

⑤

A B + C D * E / - F +

①

②

③

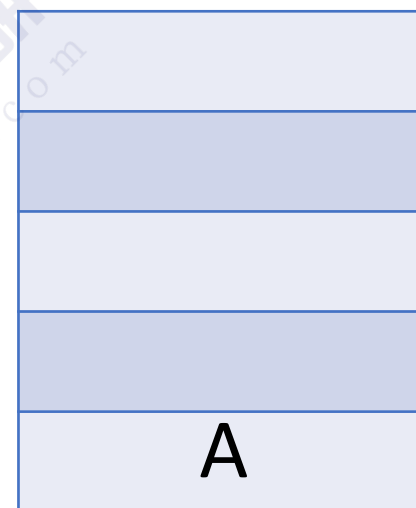
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F



④

②

③

⑤

A B + C D * E / - F +

①

②

③

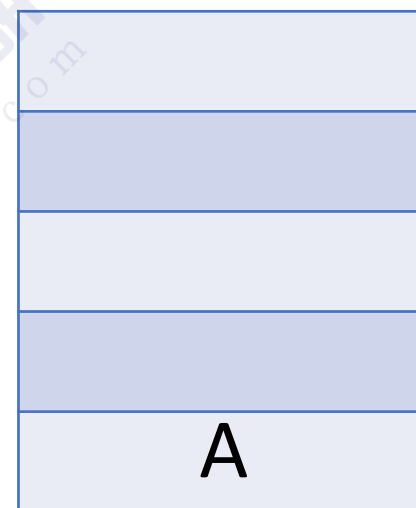
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

① ↑ ④ ② ③ ⑤

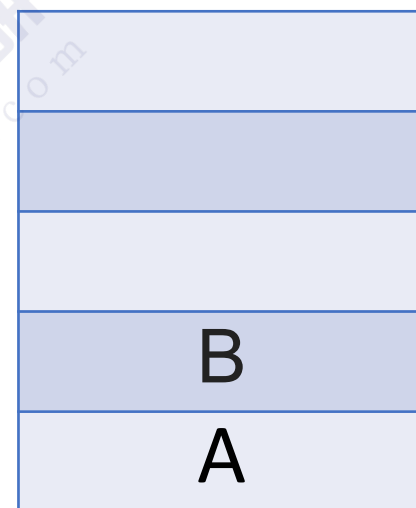
A B + C D * E / - F +

① ② ③ ④ ⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

①



②

③

⑤

A B + C D * E / - F +

①

②

③

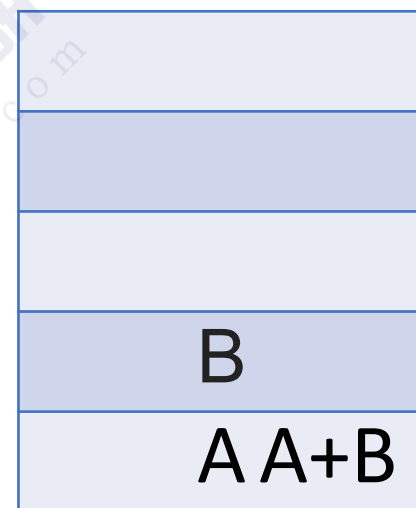
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

①

④

②

③

⑤

A B + C D * E / - F +

①

②

③

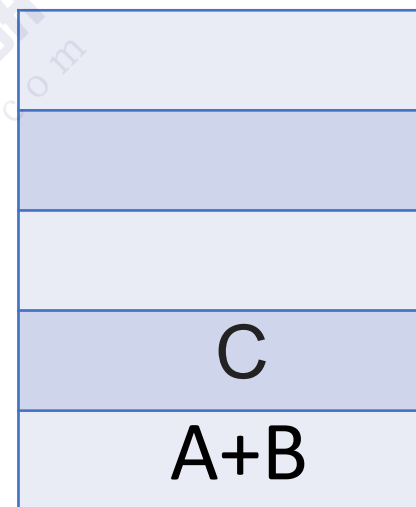
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

①

④



③

⑤

A B + C D * E / - F +

①

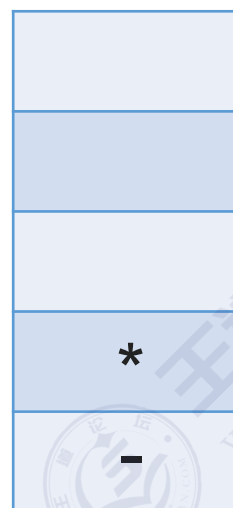
②

③

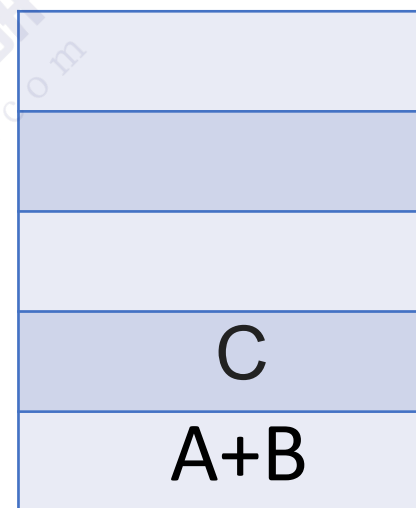
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

①

④

②

③

⑤



A B + C D * E / - F +

①

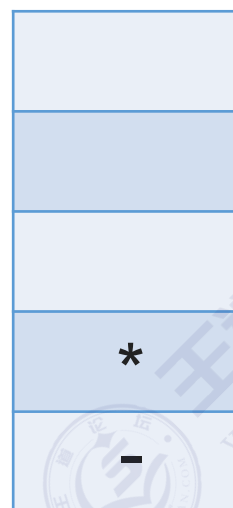
②

③

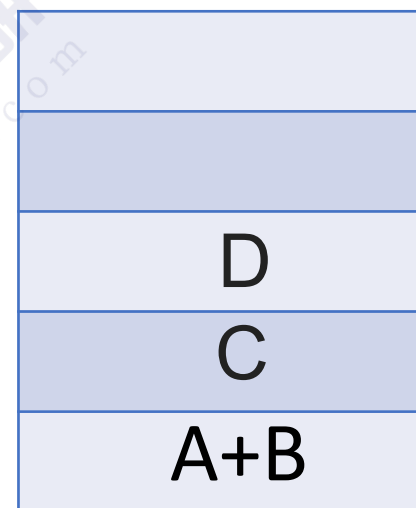
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

①

④

②



⑤

A B + C D * E / - F +

①

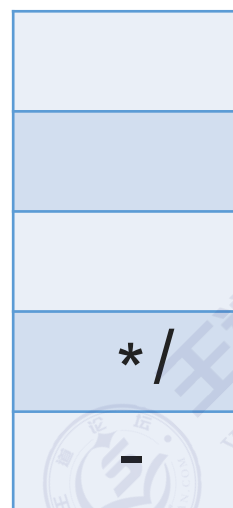
②

③

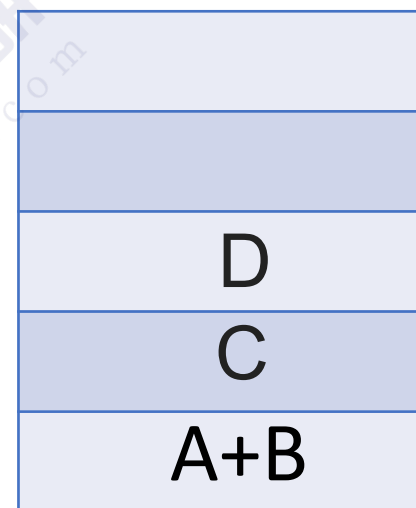
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

①

④

②

③

⑤



A B + C D * E / - F +

①

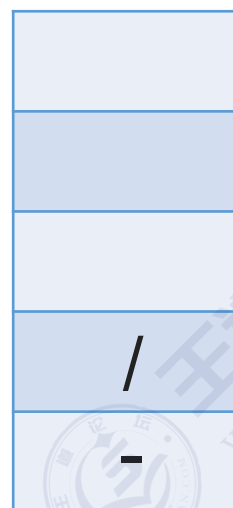
②

③

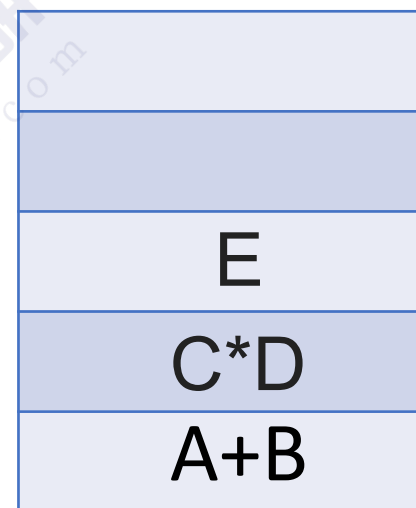
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

①

④

②

③



A B + C D * E / - F +

①

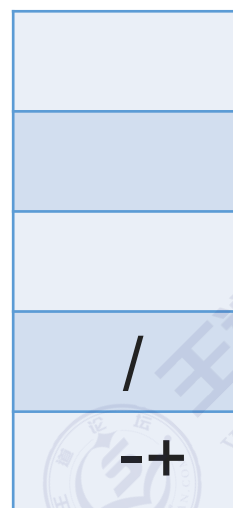
②

③

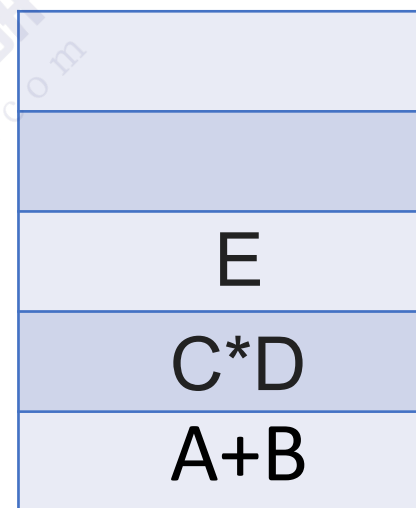
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

A + B - C * D / E + F

①

④

②

③

⑤



A B + C D * E / - F +

①

②

③

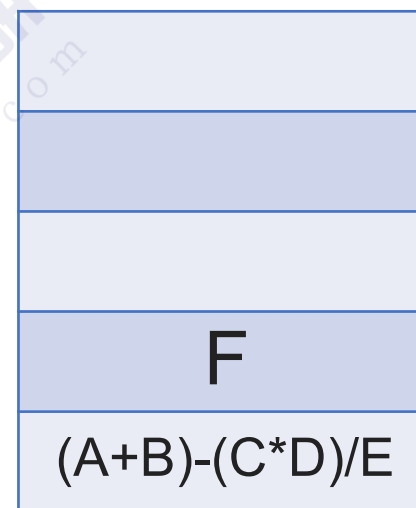
④

⑤

运算符栈



操作数栈



中缀表达式的计算（用栈实现）

用栈实现中缀表达式的计算：

初始化两个栈，操作数栈和运算符栈

若扫描到操作数，压入操作数栈

若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）



就我这小暴脾气哟~

关于Why的思考：搞这么复杂有毛意义？

A + B - C * D / E + F

①

④

②

③

⑤



A B + C D * E / - F +

①

②

③

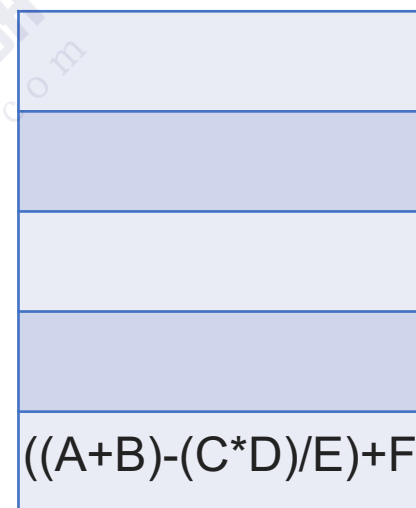
④

⑤

运算符栈



操作数栈



知识回顾与重要考点

用栈实现中缀表达式转后缀表达式:

初始化一个栈, 用于保存暂时还不能确定运算顺序的运算符。

从左到右处理各个元素, 直到末尾。可能遇到三种情况:

- ① 遇到**操作数**。直接加入后缀表达式。
- ② 遇到**界限符**。遇到“(”直接入栈; 遇到”)”则依次弹出栈内运算符并加入后缀表达式, 直到弹出“(”为止。注意:“(”不加入后缀表达式。
- ③ 遇到**运算符**。依次弹出栈中优先级高于或等于当前运算符的所有运算符, 并加入后缀表达式, 若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后, 将栈中剩余运算符依次弹出, 并加入后缀表达式。

用栈实现后缀表达式的计算:

- ① 从左往右扫描下一个元素, 直到处理完所有元素
- ② 若扫描到操作数则压入栈, 并回到①; 否则执行③
- ③ 若扫描到运算符, 则弹出两个栈顶元素, 执行相应运算, 运算结果压回栈顶, 回到①

用栈实现中缀表达式的计算:

初始化两个栈, **操作数栈**和**运算符栈**

若扫描到操作数, 压入操作数栈

若扫描到运算符或界限符, 则按照“中缀转后缀”相同的逻辑压入运算符栈(期间也会弹出运算符, 每当弹出一个运算符时, 就需要再弹出两个操作数栈的栈顶元素并执行相应运算, 运算结果再压回操作数栈)

本节内容

栈的应用

——递归

函数调用背后的过程



```
void main() {  
    int a, b, c;  
    ...  
    func1 (a, b);  
#1 → c=a+b;  
    ...  
}
```

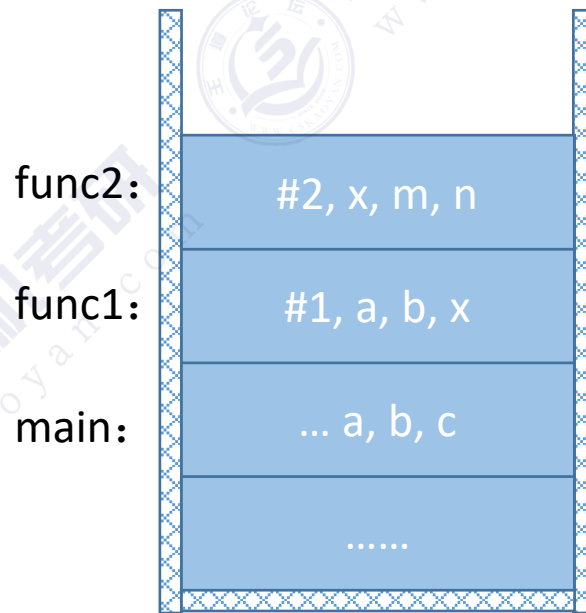
```
void func1 (int a, int b) {  
    int x;  
    ...  
    func2 (x);  
#2 → x=x+10086;  
    ...  
}
```

```
void func2 (int x) {  
    int m, n;  
    ...  
}
```

函数调用的特点：最后被调用的函数最先执行结束（LIFO）

函数调用时，需要用栈存储：

- ① 调用返回地址
- ② 实参
- ③ 局部变量



函数调用栈

函数调用背后的过程

```
170 int func2 (int x) {  
171     int m, n;  
172     m = x + 1;  
173     n = x + 2;  
174 }  
175  
176 int func1 (int a, int b) { a: 1 b: 2  
177     int x= a+b; x: 3  
178     func2 (x);  
179     x = x+10086;  
180 }  
181  
182 int main() {  
183     int a = 1, b = 2, c = 3;  
184     func1(a, b);  
185     c = a+b;  
186 }
```

The image displays three screenshots of a debugger's Stack window, illustrating the state of the call stack at different points in the program's execution.

Top Screenshot: The stack shows the current frame `func2(int) main.cpp:173` selected. The variables panel shows `x = {int} 3`, `m = {int} 4`, and `n = {int} 0`. The frames list includes `func1(int, int) main.cpp:178`, `main main.cpp:184`, and `start 0x00007fff713d13d5`.

Middle Screenshot: The stack shows the current frame `func1(int, int) main.cpp:178` selected. The variables panel shows `a = {int} 1`, `b = {int} 2`, and `x = {int} 3`. The frames list includes `func2(int) main.cpp:173`, `main main.cpp:184`, and `start 0x00007fff713d13d5`.

Bottom Screenshot: The stack shows the current frame `main main.cpp:184` selected. The variables panel shows `a = {int} 1`, `b = {int} 2`, and `c = {int} 3`. The frames list includes `func2(int) main.cpp:173`, `func1(int, int) main.cpp:178`, and `start 0x00007fff713d13d5`.

栈在递归中的应用

适合用“递归”算法解决：可以把原始问题转换为属性相同，但规模较小的问题

Eg 1: 计算正整数的阶乘 $n!$

$$\text{factorial}(n) = \begin{cases} n * \text{factorial}(n-1), & n > 1 \\ 1, & n = 1 \\ 1, & n = 0 \end{cases}$$

递归表达式
(递归体)

边界条件
(递归出口)

Eg 2: 求斐波那契数列

$$\text{Fib}(n) = \begin{cases} \text{Fib}(n-1) + \text{Fib}(n-2), & n > 1 \\ 1, & n = 1 \\ 0, & n = 0 \end{cases}$$

栈在递归中的应用

Eg 1: 递归算法求阶乘

```
182 //计算正整数 n!
183 int factorial (int n){
184     if (n==0 || n==1)
185         return 1;
186     else
187         return n*factorial(n-1);
188 }
189
190 int main() {
191     //... 其他代码
192     int x=factorial(10);
193     printf("奥利给! ");
194 }
```

再次思考：递归算法的空间复杂度

(第10层) :

(第9层) :

(第8层) :

(第7层) :

(第6层) :

(第5层) :

(第4层) :

(第3层) :

递归函数factorial (第2层) :

递归函数factorial (第1层) :

main:

#187, n=1

#187, n=2

#187, n=3

#187, n=4

#187, n=5

#187, n=6

#187, n=7

#187, n=8

#187, n=9

#192, n=10

... x

.....

函数调用栈

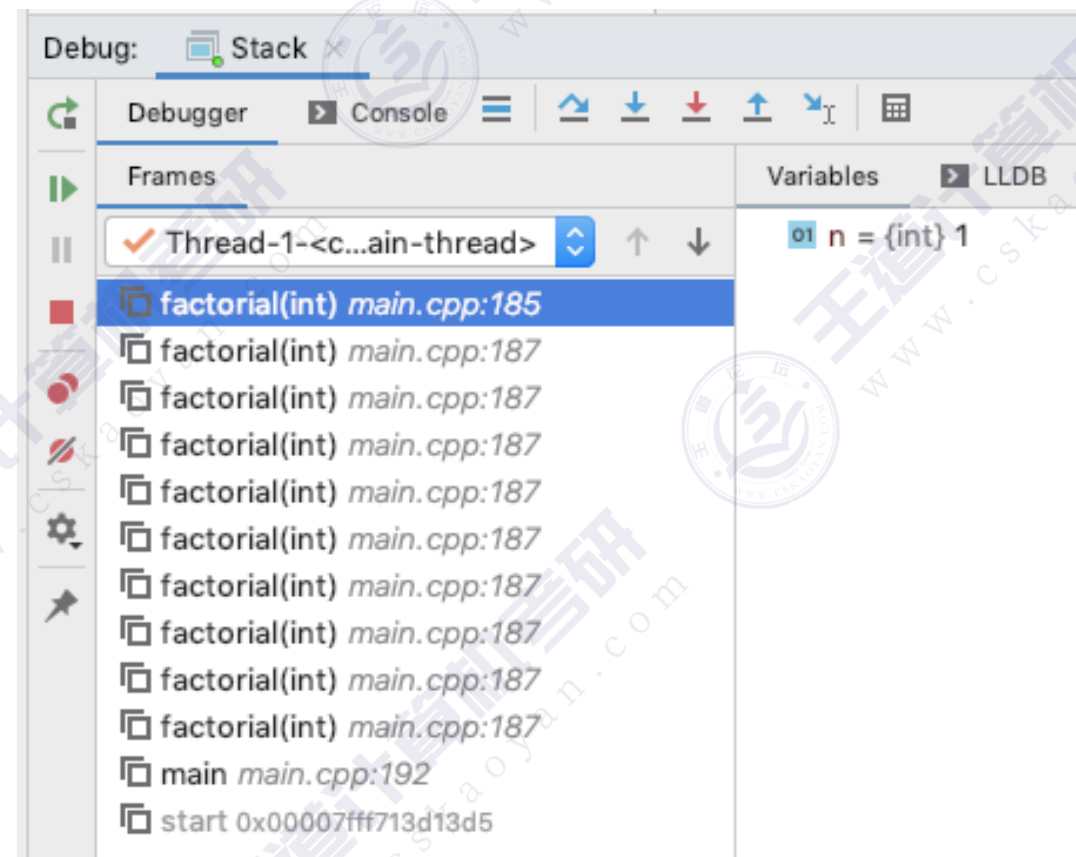
递归调用时，函数调用栈可称为“递归工作栈”
每进入一层递归，就将递归调用所需信息压入栈顶
每退出一层递归，就从栈顶弹出相应信息

缺点：太多层递归可能会导致栈溢出

栈在递归中的应用

Eg 1: 递归算法求阶乘

```
182 //计算正整数 n!  
183 int factorial (int n){  
184     if (n==0 || n==1)  
185         return 1;  
186     else  
187         return n*factorial(n-1);  
188 }  
189  
190 int main() {  
191     //... 其他代码  
192     int x=factorial(10);  
193     printf("奥利给!");  
194 }
```

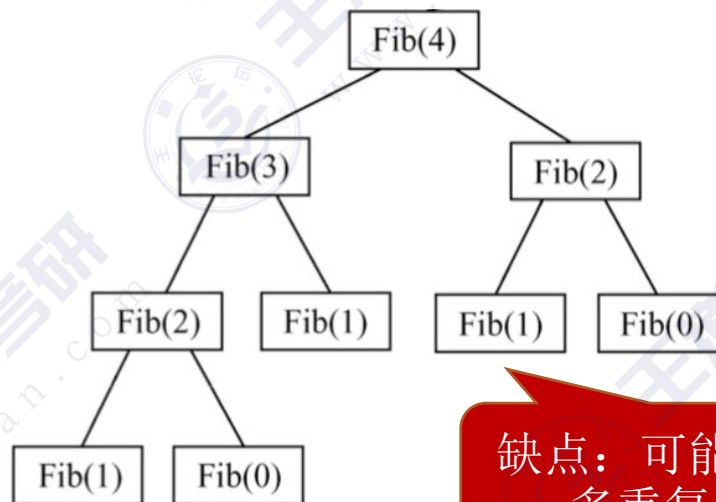


可以自定义栈将递归算法改造成非递归算法

栈在递归中的应用

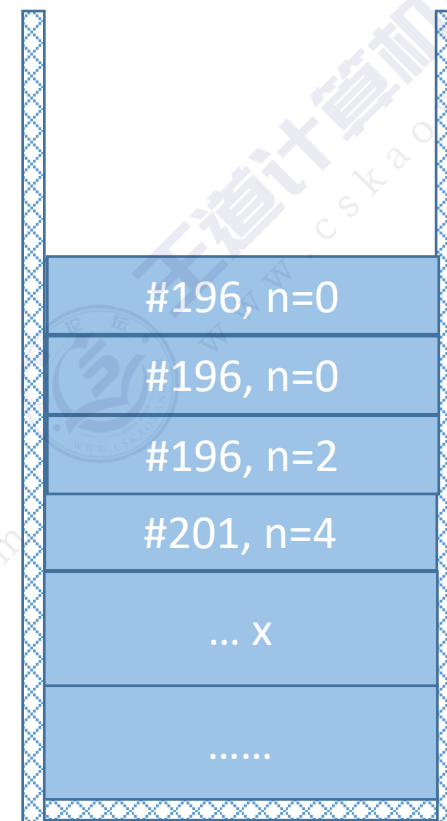
Eg 2: 递归算法求斐波那契数列

```
190 int Fib(int n){
191     if(n==0)
192         return 0;
193     else if(n==1)
194         return 1;
195     else
196         return Fib(n-1)+Fib(n-2);
197 }
198
199 int main() {
200     //... 其他代码
201     int x=Fib(4);
202     printf("奥利给! ");
203 }
```



缺点：可能包含很多重复计算

(第4层): #196, n=0
(第3层): #196, n=0
(第2层): #196, n=2
Fib (第1层): #201, n=4
main: ... x
.....



函数调用栈

知识回顾与重要考点

函数调用的特点：最后被调用的函数最先执行结束（LIFO）

函数调用时，需要用“函数调用栈”存储：

- ① 调用返回地址
- ② 实参
- ③ 局部变量

递归调用时，函数调用栈可称为“递归工作栈”

每进入一层递归，就将递归调用所需信息压入栈顶

每退出一层递归，就从栈顶弹出相应信息

缺点：效率低，太多层递归可能会导致栈溢出；可能包含很多重复计算

可以自定义栈将递归算法改造成非递归算法

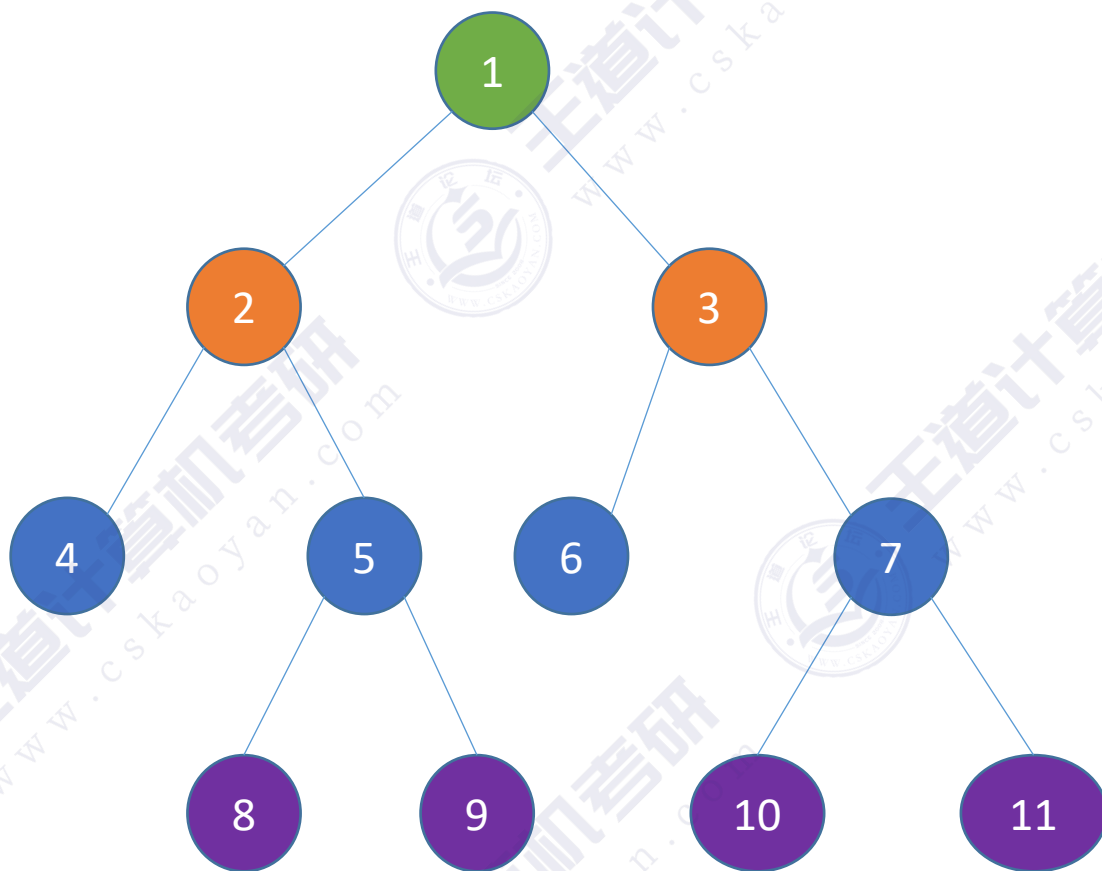
本节内容

队列 的应用

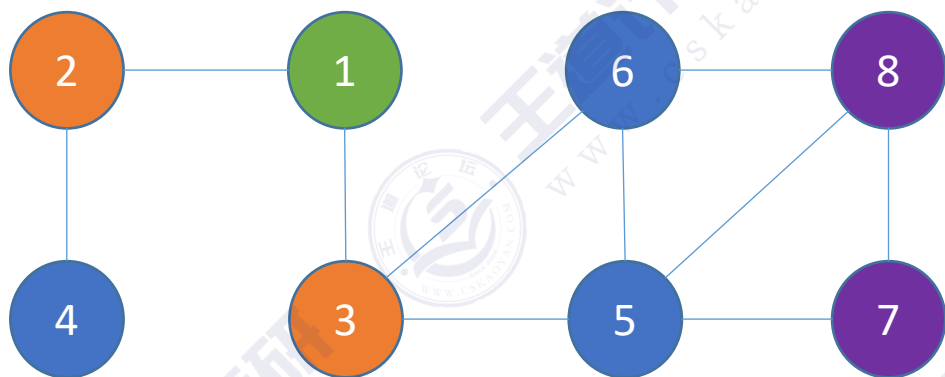
队列应用——树的层次遍历



注：在“树”章节中会详细学习



队列应用——图的广度优先遍历



注：在“图”章节中会详细学习

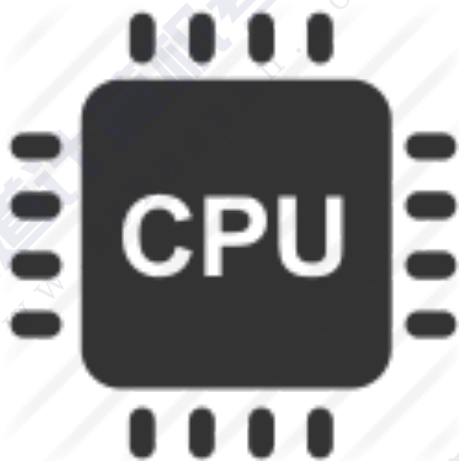
队列在操作系统中的应用

多个进程争抢着使用有限的系统资源时，**FCFS**（First Come First Service，**先来先服务**）是一种常用策略。

可用队列实现

Eg: CPU资源的分配

就绪进程队列



队列在操作系统中的应用

多个进程争抢着使用有限的系统资源时，**FCFS**（First Come First Service，**先来先服务**）是一种常用策略。

可用队列实现

Eg: 打印数据缓冲区

场景：去学校打印店打印论文，多个同学用同一台打印机打印，打印的先后顺序如何？

可缓解主机与打印机速度不匹配的问题

缓冲区（用“队列”组织打印数据）

